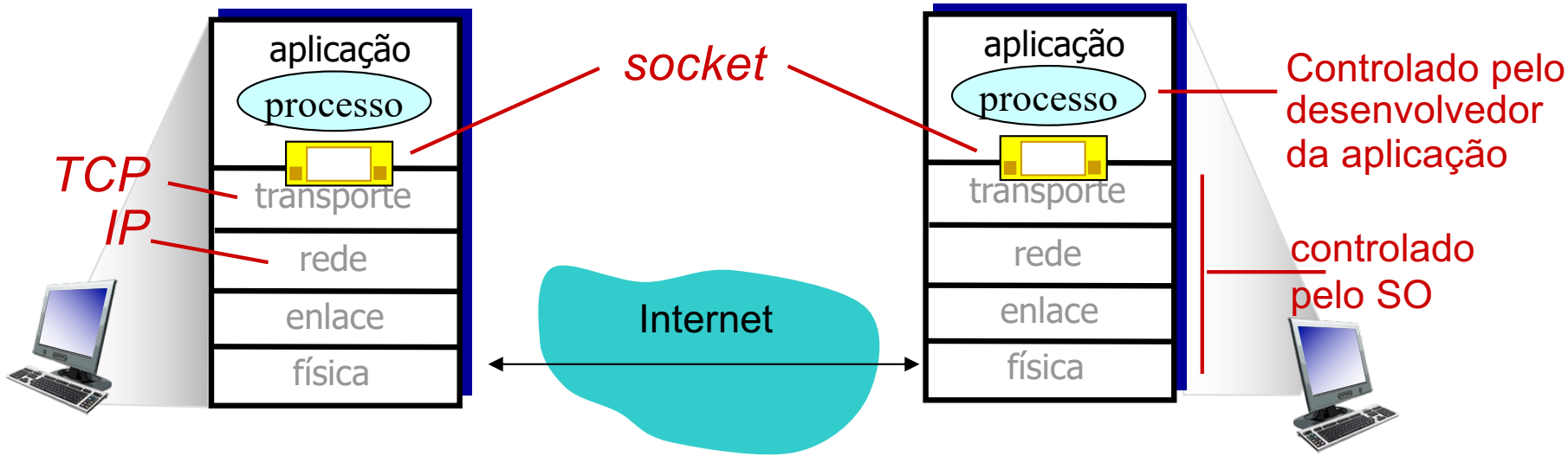
Sockets - Pilha TCP/IP

- Os processos enviam/ recebem mensagens para/dos seus *sockets*
- *Um socket* é análogo a uma porta
  - Processo transmissor envia a mensagem através da porta
  - O processo transmissor assume a existência da infraestrutura de transporte no outro lado da porta que faz com que a mensagem chegue ao *socket* do processo receptor



Endereçamento de processos

- Para que um processo receba mensagens, ele deve possuir um identificador
- Cada host possui um endereço IP único
- P: o endereço IP do host no qual o processo está sendo executado é suficiente para identificar o processo?
- Resposta: Não, muitos processos podem estar executando no mesmo host
- O identificador inclui tanto o endereço IP quanto os números das portas associadas com o processo no host .
- Exemplo de números de portas:
  - Servidor HTTP: 80
  - Servidor de Correio: 25
- Para enviar uma msg HTTP para o servidor Web gaia.cs.umass.edu
  - Endereço IP: 128.119.245.12
  - Número da porta: 80

Endereçamento de processos

Macintosh HD — -bash — 85×19

```
MacBook-Air-de-Marco:/ birchal$ netstat
Active Internet connections
Proto Recv-Q Send-Q  Local Address          Foreign Address        (state)
tcp4       0      0  localhost.49219        localhost.51907        ESTABLISHED
tcp4       0      0  localhost.51907        localhost.49219        ESTABLISHED
tcp4       0      0  10.250.89.233.51904    162.125.5.7.https      ESTABLISHED
tcp4       0      0  10.250.89.233.51903    162.125.5.3.https      ESTABLISHED
tcp4     528      0  10.250.89.233.51883    104.19.198.151.https   ESTABLISHED
tcp4      31      0  10.250.89.233.51859    162.125.34.6.https     CLOSE_WAIT
tcp4       0      0  10.250.89.233.51808    162.125.18.133.https   ESTABLISHED
tcp4       0      0  10.250.89.233.51801    17.188.138.71.5223     ESTABLISHED
tcp4       0      0  10.250.89.233.51800    17.249.53.28.5223      ESTABLISHED
tcp4       0      0  10.250.89.233.51789    172.217.192.188.https  ESTABLISHED
tcp4       0      0  localhost.49264        localhost.51768        ESTABLISHED
tcp4       0      0  localhost.51768        localhost.49264        ESTABLISHED
tcp4       0      0  localhost.49264        localhost.51553        ESTABLISHED
tcp4       0      0  localhost.51553        localhost.49264        ESTABLISHED
tcp4       0      0  localhost.49264        localhost.51474        CLOSE_WAIT
tcp4       0      0  localhost.49264        localhost.51216        CLOSE_WAIT
```

Serviços providos pelos protocolos de transporte da Internet

Serviço TCP:

- *transporte confiável* entre processos remetente e receptor
- *controle de fluxo:* remetente não vai "afogar" receptor
- *controle de congestionamento:* estrangular remetente quando a rede estiver carregada
- *não provê:* garantias temporais ou de banda mínima
- *orientado a conexão:* apresentação requerida entre cliente e servidor

Serviço UDP:

- transferência de dados não confiável entre processos remetente e receptor
- não provê: estabelecimento da conexão, confiabilidade, controle de fluxo, controle de congestionamento, garantias temporais ou de banda mínima

P: Qual é o interesse em ter um protocolo como o UDP?

Programação Socket

Dois tipos de sockets para dois tipos de serviço de transporte:

- *UDP:* datagrama não confiável
- *TCP:* stream de bytes orientada à conexão e confiável

Applicação Exemplo:

1. O cliente lê uma linha de caracteres (dados) do teclado e os envia ao servidor
2. O servidor recebe os dados e converte para letras maiúsculas
3. O servidor envia os dados modificados para o cliente
4. O cliente recebe os dados modificados e os exibe em sua tela

Programação Socket com UDP

UDP: Não há conexão entre cliente e servidor

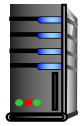
- sem handshaking antes do envio de dados
- O transmissor explicitamente anexa o IP e Porta do destino em cada pacote
- O receptor extrai o endereço IP e porta do pacote recebido

UDP: Os dados podem ser perdidos ou recebidos fora de ordem

Ponto de vista da aplicação:

- UDP prove transferência não confiável de conjuntos de dados (datagramas) entre os processos cliente e servidor.

Interação cliente/servidor socket: UDP



server (running on serverIP)

create socket, port= x:
serverSocket =
socket(AF_INET,SOCK_DGRAM)

read datagram from
serverSocket

write reply to
serverSocket
specifying
client address,
port number

client

create socket:
clientSocket =
socket(AF_INET,SOCK_DGRAM)

Create datagram with serverIP address
And port=x; send datagram via
clientSocket

read datagram from
clientSocket

close
clientSocket

Aplicação exemplo: UDP client

Python UDPClient

include Python's socket library →
create UDP socket →
get user keyboard input →
attach server name, port to message; send into socket →
read reply data (bytes) from socket →
print out received string and close socket →

```
from socket import *
serverName = 'hostname'
serverPort = 12000
clientSocket = socket(AF_INET,
                      SOCK_DGRAM)
message = input('Input lowercase sentence:')
clientSocket.sendto(message.encode(),
                    (serverName, serverPort))
modifiedMessage, serverAddress =
                    clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

Note: this code update (2023) to Python 3

Aplicação exemplo: UDP server

Python UDPServer

create UDP socket →
bind socket to local port number 12000 →
loop forever →
Read from UDP socket into message, getting client's address (client IP and port) →
send upper case string back to this client →

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print('The server is ready to receive')
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(),
                        clientAddress)
```

Note: this code update (2023) to Python 3

Sockets - UDP - execução UNIX

birchal — Python UDPServer.py — 80×24

```
Last login: Wed Sep 18 17:22:35 on console
birchal@MacBook-Pro-de-Marco ~ % python3 UDPServer.py
The server is ready to receive
```

birchal — -zsh — 80×24

```
birchal@MacBook-Pro-de-Marco ~ % python3 UDPClient.py
 Input lowercase sentence:teste de aplicacao udp
TESTE DE APLICACAO UDP
birchal@MacBook-Pro-de-Marco ~ %
```

birchal — nc -u 127.0.0.1 12000 — 80×24

```
birchal@MacBook-Pro-de-Marco ~ % nc -u 127.0.0.1 12000
isso e um teste usando o comando nc (netcat) como cliente
ISSO E UM TESTE USANDO O COMANDO NC (NETCAT) COMO CLIENTE
```

OBS Unix: comando nc (netcat) serve tanto como cliente como servidor (parâmetro –l), TCP (default) e UDP (parâmetro –u). No exemplo é usado como cliente.

Sockets - UDP - execução Windows

Administrator: Command Pro

```
C:\redes>py UDPServer.py
The server is ready to receive
```

Administrator: Command Pro

```
C:\redes>py UDPClient.py
 Input lowercase sentence:este teste usa o clienteUDP phyton
ESTE TESTE USA O CLIENTEUDP PHYTON

C:\redes>
```

Administrator: Command Pro

```
C:\>ncat -u 127.0.0.1 12000
isso e um teste usando ncat como cliente udp
ISSO E UM TESTE USANDO NCAT COMO CLIENTE UDP
```

Windows: ncat (parte do programa nmap) é similar ao nc (netcat)

Programação Socket com TCP

O cliente precisa conectar no servidor

- O servidor precisa já estar rodando
- O servidor precisa ter aberto uma porta para aceitar a conexão do cliente

Como o cliente conecta ao servidor:

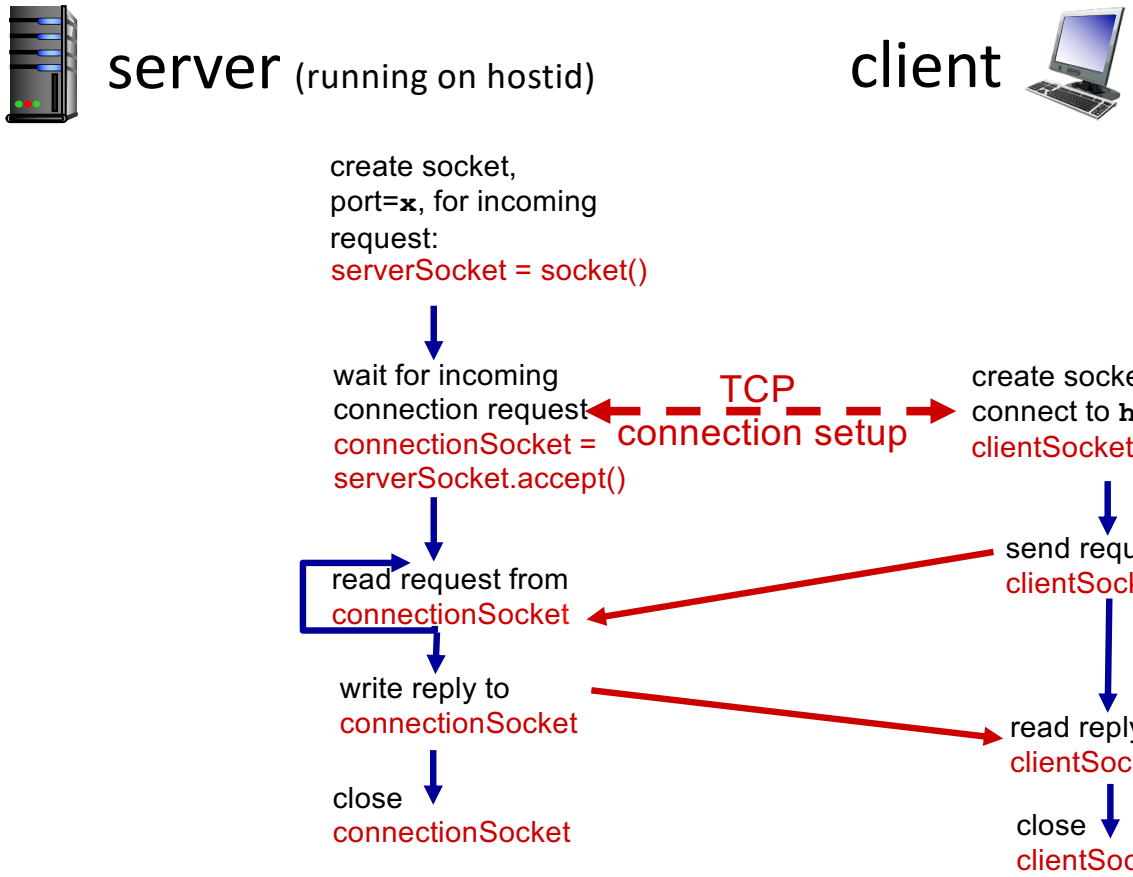
- Cria um socket TCP especificando o IP e a porta do servidor
- *Quando o cliente cria o socket,* o cliente TCP estabelece a conexão com o servidor TCP

- Quando contactado por um cliente, *o servidor TCP cria um novo socket* para que o processo servidor possa comunicar com aquele cliente em particular
  - Isso permite que um servidor TCP possa conversar com múltiplos clientes
  - O IP e a porta do cliente são usados para distinguir cada um dos clientes

Ponto de Vista da Aplicação

TCP prove transferência orientada à conexão (em ordem) e confiável para transferência de fluxos de bytes ("pipe") entre os processo cliente e servidor

Interação cliente/servidor socket: TCP



Aplicação exemplo: TCP client

Python TCPClient

create TCP socket for server, remote port 12000

No need to attach server name, port

```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName,serverPort))
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ('From Server:', modifiedSentence.decode())
clientSocket.close()
```

Note: this code update (2023) to Python 3

Aplicação exemplo: TCP server

Python TCPServer

create TCP welcoming socket

server begins listening for incoming TCP requests

loop forever

server waits on accept() for incoming requests, new socket created on return

read bytes from socket (but not address as in UDP)

close connection to this client (but *not* welcoming socket)

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
print('The server is ready to receive')
while True:
    connectionSocket, addr = serverSocket.accept()

    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.
                                        encode())
    connectionSocket.close()
```

Note: this code update (2023) to Python 3

Sockets - TCP - execução UNIX

birchal — Python TCPServer.py — 80×11

```
birchal@MacBook-Pro-de-Marco ~ % python3 TCPServer.py
The server is ready to receive
```

birchal — -zsh — 80×24

```
birchal@MacBook-Pro-de-Marco ~ % python3 TCPClient.py
Input lowercase sentence:este e um teste TCP cliente
From Server: ESTE E UM TESTE TCP CLIENTE
birchal@MacBook-Pro-de-Marco ~ %
```

birchal — -zsh — 80×24

```
birchal@MacBook-Pro-de-Marco ~ % telnet 127.0.0.1 12000
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
teste usando telnet como cliente TCP
TESTE USANDO TELNET COMO CLIENTE TCP
Connection closed by foreign host.
birchal@MacBook-Pro-de-Marco ~ %
birchal@MacBook-Pro-de-Marco ~ %
birchal@MacBook-Pro-de-Marco ~ %
birchal@MacBook-Pro-de-Marco ~ % nc 127.0.0.1 12000
teste usando nc (netcat) como cliente TCP
TESTE USANDO NC (NETCAT) COMO CLIENTE TCP
birchal@MacBook-Pro-de-Marco ~ %
```

Sockets - TCP - execução Windows

Administrator: Command Pro

```
C:\redes>py TCPServer.py
The server is ready to receive
```

Administrator: Command Pro

```
C:\redes>py TCPClient.py
Input lowercase sentence:este teste usa cliente TCP phyton
From Server: ESTE TESTE USA CLIENTE TCP PHYTON
```

Administrator: Command Pro

```
C:\redes>ncat 127.0.0.1 12000
teste ncat como cliente tcp
TESTE NCAT COMO CLIENTE TCP
```

Sockets - TCP - execução Windows

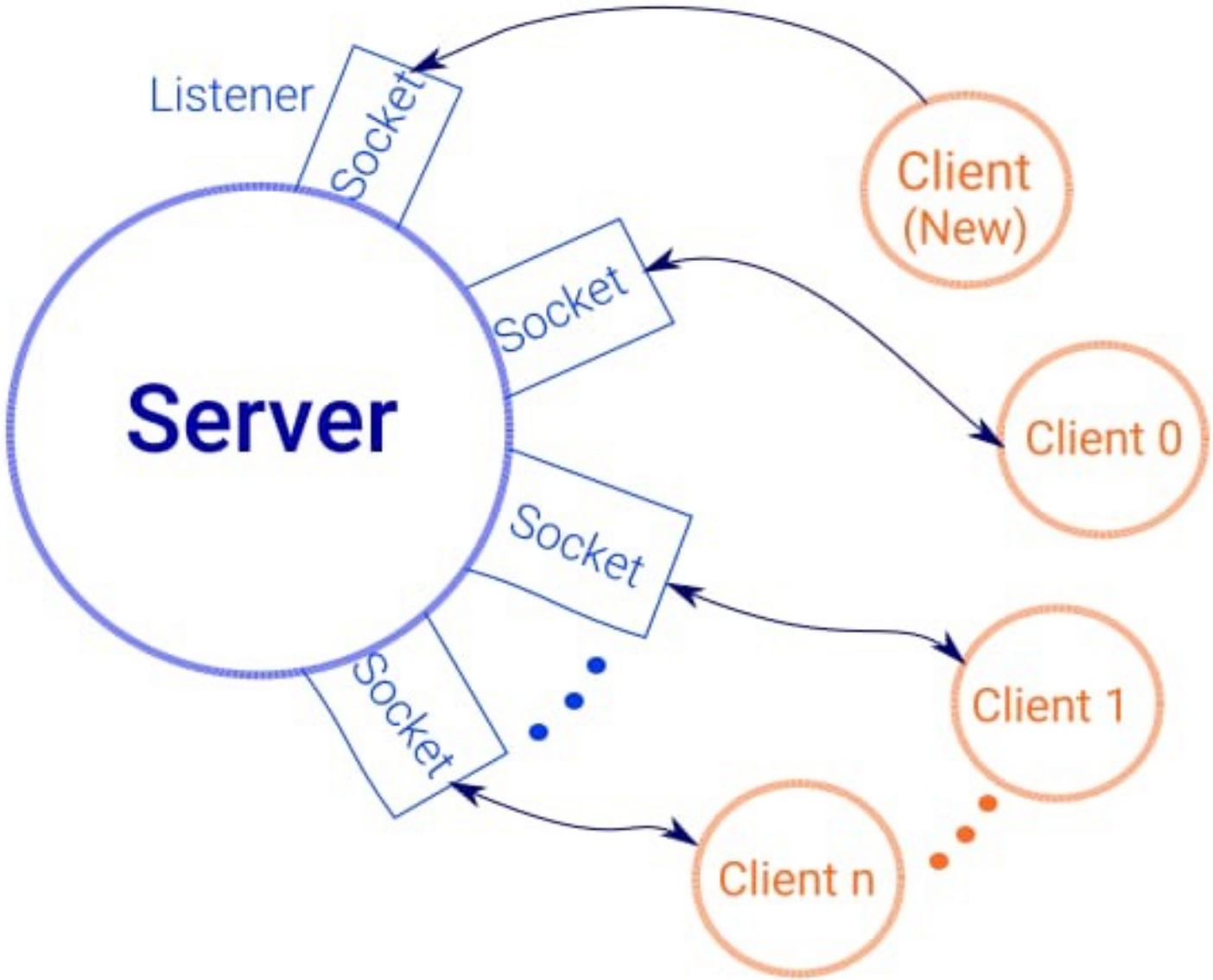
```
Administrator: Command Pro

Welcome to Microsoft Telnet Client

Escape Character is 'CTRL+]'

Microsoft Telnet> set localecho
Local echo on
Microsoft Telnet> open 127.0.0.1 12000


 A

Connection to host lost.

C:\redes>
```

Programação Socket com TCP Concorrente

Concurrent Servers → Vários clients podem conectar no servidor

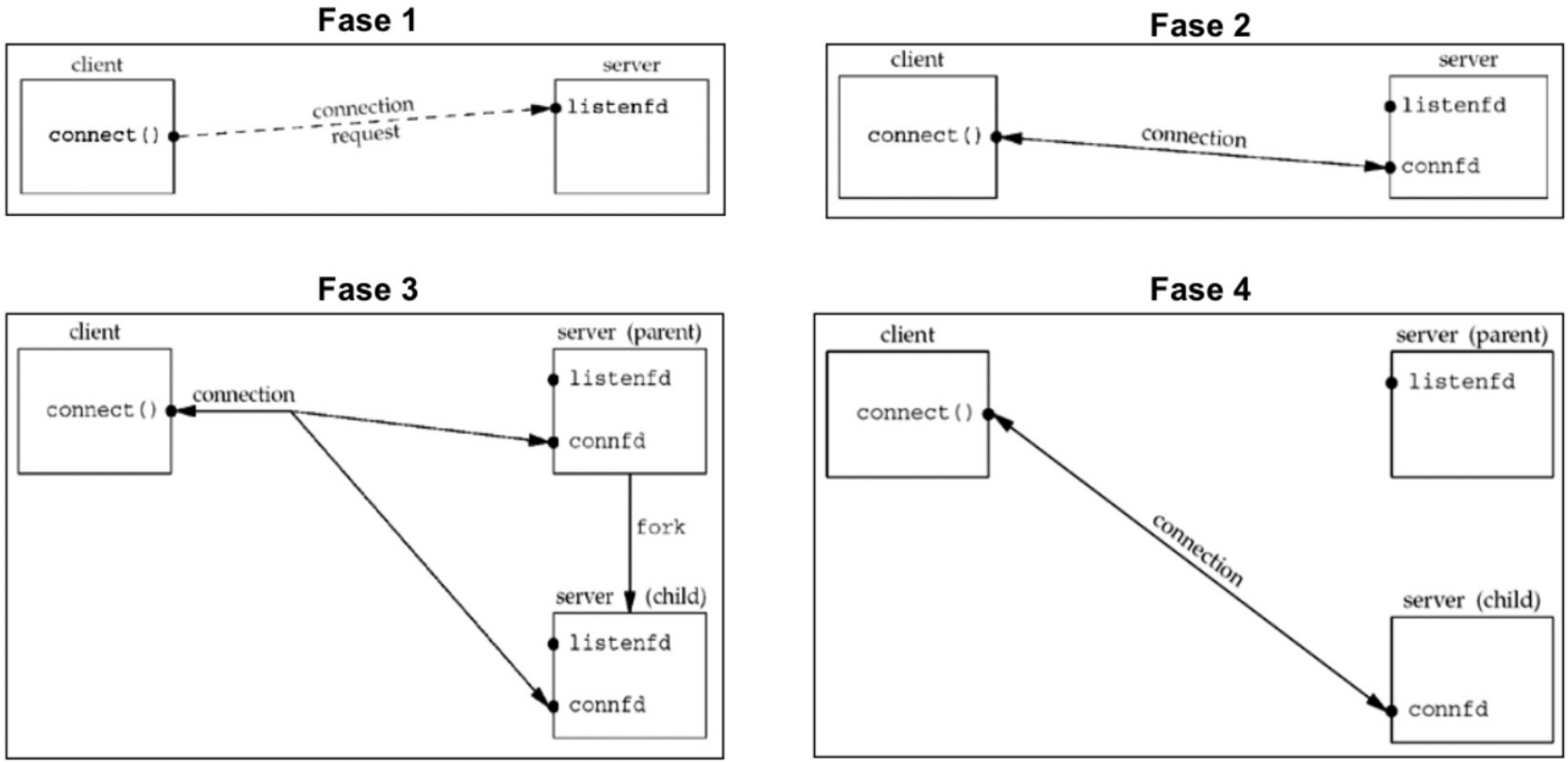
- Num ambiente com múltiplos clientes, o tratamento sequencial das mensagens não é o mais eficiente.
- A prática mais recomendada é a de criar um servidor concorrente.
- O servidor concorrente inicia com uma thread principal que cria e escuta o Listener Socket.
- Quando um cliente se conecta, a thread principal cria uma nova thread de trabalho (ou processo) para cada conexão estabelecida, criando nesta thread, um novo socket que lida com o envio e recebimento de mensagens para o cliente.
- Cada cliente terá seu próprio socket de comunicação com o servidor e este volta a aguardar um novo cliente no *Listener Socket*.

Programação Socket com TCP Concorrente



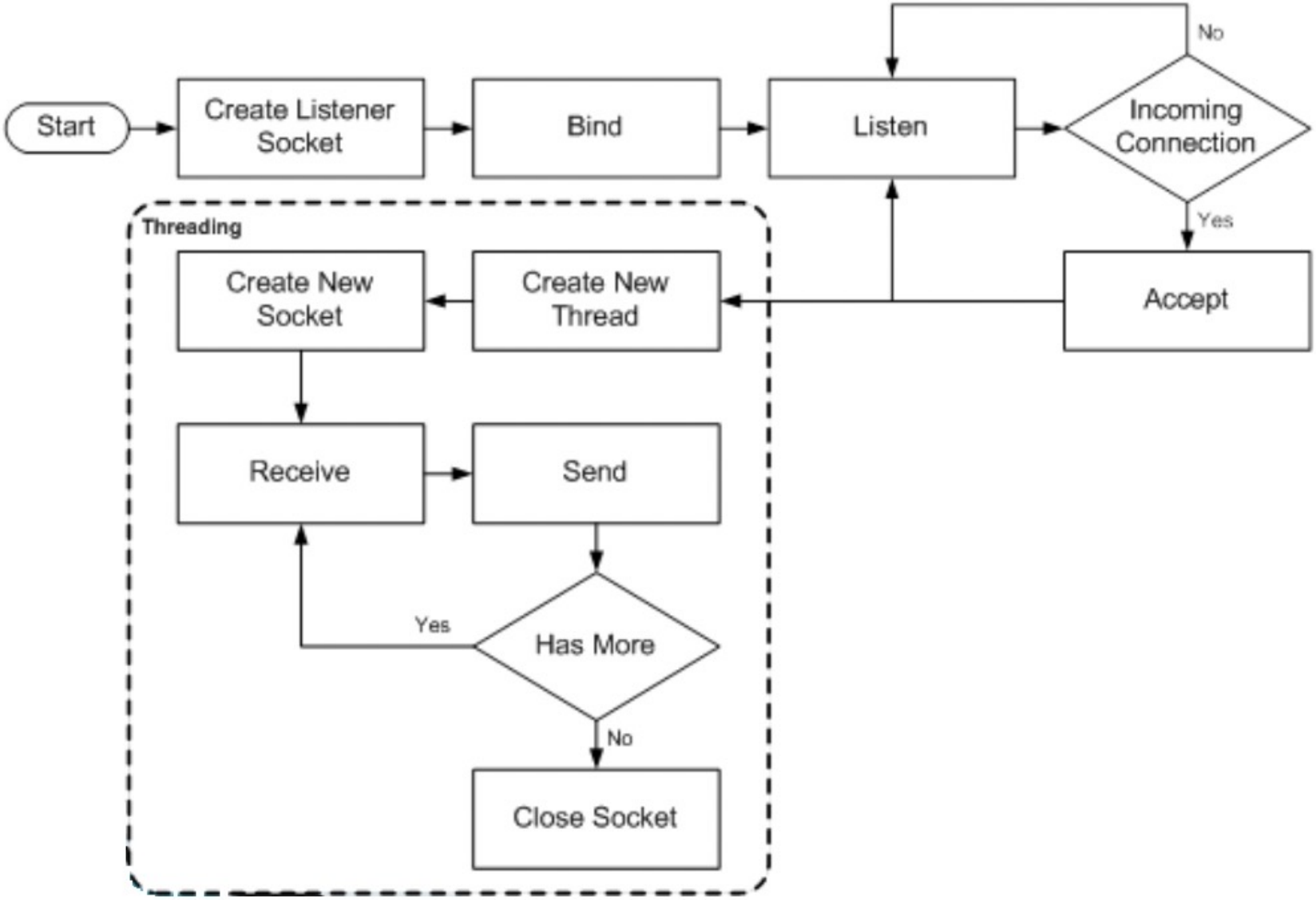
Server and Clients

Programação Socket com TCP Concorrente



Unix Network Programming, 3rd edition, W. Richard Stevens, Bill Fenner & Andrew M. Rudoff, Addison-Wesley, 2003

Programação Socket com TCP Concorrente Servidor



Programação Socket com TCP Concorrente Servidor

```
# -*- coding: utf-8 -*-
'''
Exemplo de um Servidor TCP Concorrente (Multithread)

Artigo: https://www.linkedin.com/pulse/python-sockets-criando-um-servidor-tcp-concorrente-
diego/

Diego Mendes Rodrigues
'''

import socket
import _thread

HOST = '127.0.0.1' # Endereco IP do Servidor
PORT = 50000 # Porta que o Servidor está
```

Programação Socket com TCP Concorrente Servidor

```
# Função chamada quando uma nova thread for iniciada
def conectado(con, cliente):
        print('\nCliente conectado:', cliente)

        while True:
                # Recebendo as mensagens através da conexão
                msg = con.recv(1024)
                if not msg:
                        break

                print('\nCliente..:', cliente)
                print('Mensagem.:', msg)

        print('\nFinalizando conexao do cliente', cliente)
        con.close()
        _thread.exit()
```

Programação Socket com TCP Concorrente Servidor

```
tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
orig = (HOST, PORT)

# Colocando um endereço IP e uma porta no Socket
tcp.bind(orig)

# Colocando o Socket em modo passivo
tcp.listen(1)
print('\nServidor TCP concorrente iniciado no IP', HOST, 'na porta', PORT)

while True:
        # Aceitando uma nova conexão
        con, cliente = tcp.accept()
        print('\nNova thread iniciada para essa conexão’)

        # Abrindo uma thread para a conexão
        _thread.start_new_thread(conectado, tuple([con, cliente]))

# Fechando a conexão com o Socket
tcp.close()
```

Programação Socket com TCP Concorrente Cliente

```
import socket

HOST = '127.0.0.1' # Endereco IP do Servidor
PORT = 50000 # Porta que o Servidor está

# Criando a conexão
tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
destino = (HOST, PORT)
tcp.connect(destino)

print('\nDigite suas mensagens')
print('Para sair use CTRL+X\n')

# Recebendo a mensagem do usuário final pelo teclado
mensagem = input()

# Enviando a mensagem para o Servidor TCP através da conexão
while mensagem != '\x18':
       tcp.send(str(mensagem).encode())
       mensagem = input()
# Fechando o Socket
tcp.close()
```

Programação Socket com TCP Concorrente

birchal — Python TCPServerConc.py — 96×19

```
birchal@MacBook-Pro-de-Marco ~ % python3 TCPServerConc.py

Servidor TCP concorrente iniciado no IP 127.0.0.1 na porta 50000

Nova thread iniciada para essa conexão

Cliente conectado: ('127.0.0.1', 49881)

Cliente..: ('127.0.0.1', 49881)
Mensagem.: b'Ola do cliente 1 !!!'

Nova thread iniciada para essa conexão

Cliente conectado: ('127.0.0.1', 49882)

Cliente..: ('127.0.0.1', 49882)
Mensagem.: b'Ola do cliente 2 !!!'
```

birchal — Python TCPClientConc.py — 58×9

```
birchal@MacBook-Pro-de-Marco ~ % python3 TCPClientConc.py

Digite suas mensagens
Para sair use CTRL+X

Ola do cliente 1 !!
```

birchal — Python TCPClientConc.py — 59×9

```
Last login: Thu Oct  3 00:58:13 on ttys003
birchal@MacBook-Pro-de-Marco ~ % python3 TCPClientConc.py

Digite suas mensagens
Para sair use CTRL+X

Ola do cliente 2 !!!
```